Laboratory 8

Mod 10 Counter

8.1 Outcomes and Objectives

The outcome of this lab is to simulate a mod10 counter that will be used as a building block in the implementation of a digital stopwatch. Through this process, you will achieve the following learning objectives:

- Designing glue logic to interface building blocks
- Design a circuit that performs register transfer
- Creating a simulation timing diagram for a module
- Creating a Do file to automate waveform setup

8.2 Module: `mod10Counter`

A mod 10 counter counts up from 0 to 9 and then rolls over back to 0 to count up again. The term "mod" comes from the word modulus. If you take a number x and form "x mod 10" you get the integer remainder after division by 10. For example, 12 mod 10 is equal to 2 because $12/10 = 1$ with a remainder of 2. Note that "x mod 10" will always produce a value between 0 and 9. Thus, a mod 10 counter will count up from 0 to 9 and then back to 0 to start over again. This works in the same way you might see an unsigned integer overflow in memory.

You will build the mod 10 counter shown in Figure 8.1. The `enb` input enables the counter to count up on a rising edge of the clock. The `synch` input causes the mod10Counter to (synchronously) reset of the rising edge of the clock. The `roll` output indicated when the `currentCount` is going to roll-over from 9 to 0.

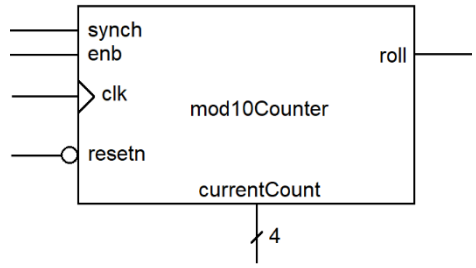

Figure 8.1: The high-level interface for the mod 10 counter.

Table 8.1 is the truth table for the `currentCount` output. When the `enb` input is at logic `1`, `currentCount` is incremented mod 10 on the next positive `clk` edge. When the `synch` input equals `1`, `currentCount` goes to 4'b0000 on the next positive `clk` edge. The `synch` input takes precedence over the `enb` input, so if both are at logic `1` then `currentCount` goes to 4'b0000 on the next positive `clk` edge.

Table 8.1: Truth table for the `currentCount` output from the `mod10Counter`.

`reset`	`clk`	`enb`	`synch`	`currentCount`	`Note`
0	x	x	x	0	Asynchronous reset
1	0, 1, ↓	x	x	`currentCount`	No clock edge
1	↑	0	0	`currentCount`	Hold
1	↑	0	1	0	Synchronous reset
1	↑	1	0	(`currentCount` + 1) mod 10	Count up
1	↑	1	1	0	Synchronous reset

Table 8.2 is the truth table for the `roll` output. If `enb` is logic `1` when the `currentCount` equals `9`, the `roll` output equals logic `1`. In all other cases, the `roll` output should equal logic `0`. Note, the `roll` output does not depend on the `clk`, it's a combinational logic block.

Table 8.2: Truth table for the `roll` output from the `mod10Counter`.

`enb`	`currentCount`	`roll`
1	`currentCount` ¡ 9	0
1	`currentCount` == 9	1

8.3 System Architecture

Now that you have a solid grasp of how the `mod10Counter` should work, let's turn our attention to how this is accomplished. The internal organization of the `mod10Counter` is shown in Figure 8.2.

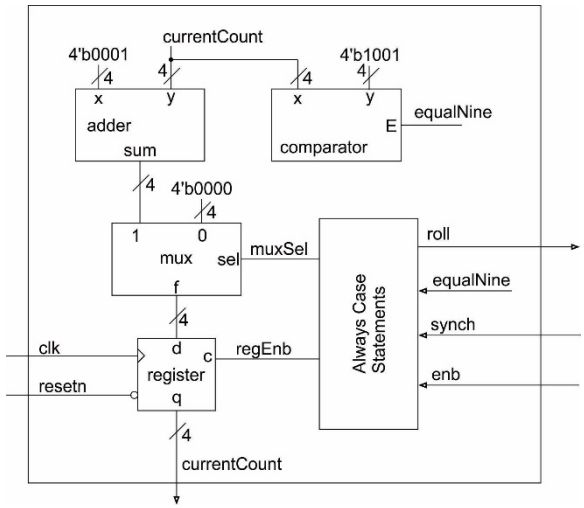

Figure 8.2: The internal organization of the `mod10counter` module.

You have been provided with the `adder`, `comparator`, `mux`, and `register` shown in Figure 8.2. In addition to wiring these building blocks together, you will need to define the logic inside the `always/case` block.

Use the truth tables in Table 8.1 and Table 8.2 along with the hardware organization in Figure 8.2 to fill in Table 8.3 .

Table 8.3: Truth table for the always/case logic inside the `mod10counter`.

enb	synch	equalNine	muxSel	regEnb	roll
0	0	0			
0	0	1	x		
0	1	0			
0	1	1			
1	0	0			0
1	0	1		1	
1	1	0			
1	1	1			

When you code your `always/case` statement create a three-bit output and then have three `assign` statements that break this 3-bit output into individual bits for `muxSel`, `regEnb` and `roll`.

Complete the coding of the `mod10Counter`, instantiating the the modules in Figure 8.2 and including the always/case statement as a CSA. Once you complete writing this code, you will need to demonstrate its proper functioning by simulating the design. You will **not** be synthesizing your design this week, so it's **imperative** that you carefully examine the output because you will be using this module in the coming weeks as an element in a larger design.

8.4 Testbench

You will need to demonstrate that your `mod10Counter` operates correctly by running the provided testbench. Since we will not be synthesizing today's module, you must be very focused on ensuring that your `mod10Counter` is operating **perfectly** because you will be using this module as a building block in several future assignments. In order for you to verify **perfect** operation, you need to understand the expected output so that you can compare it to what your simulation is actually producing. Any difference between these two indicate an error (either in your understanding or circuit behavior) that needs to be fixed.

Start by completing the timing diagram in Figure 8.3 using the value from the testbench. You will have to open the testbench to find these values.

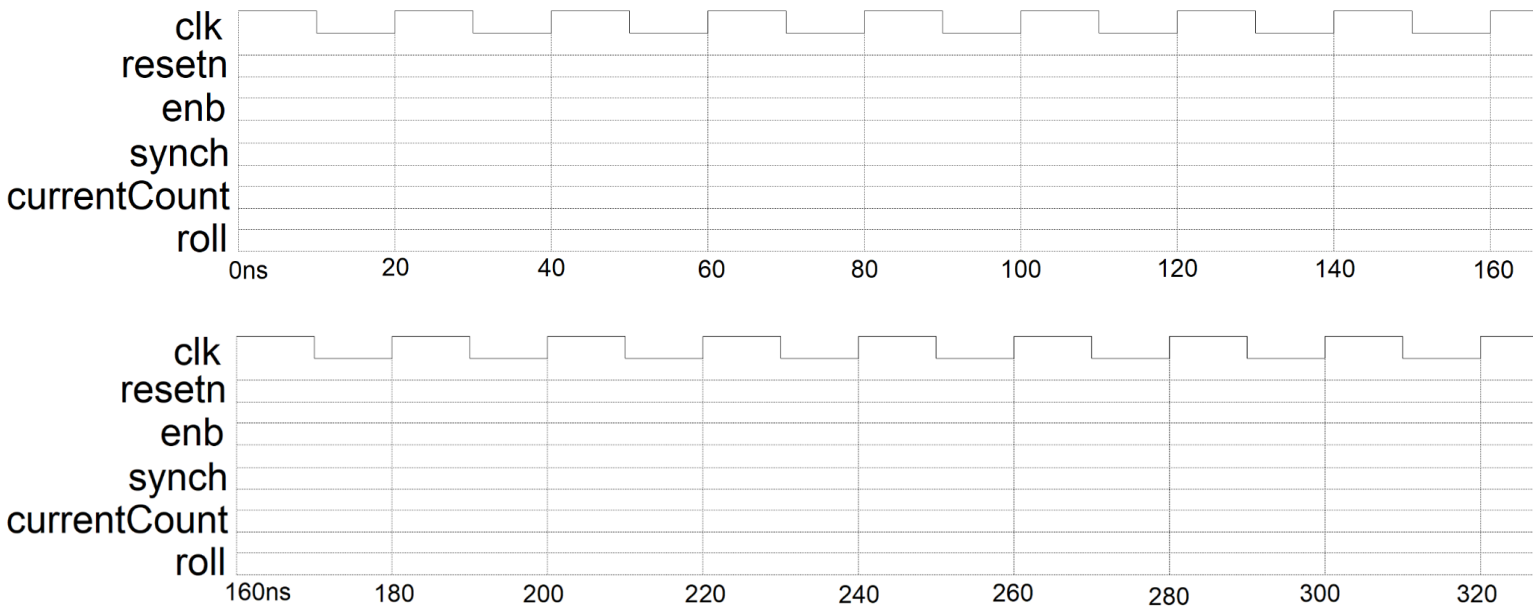

Figure 8.3: A timing diagram for you to fill out based on the testbench code for the `mod10Counter`. Note the diagram was too wide to be readable in one line, so it has been broken at 160ns to make it easier to you to read.

Listing 8.1: A partial do file for the mod10counter.

```
##########################################
# Search Internet ``Questa command reference manual''
##########################################
vsim work.mod10counter_tb
restart -f
delete wave *

add wave -position end  sim:/mod10counter_tb/uut/clk
        <<you need more add waves here>>
add wave -position end  -radix unsigned -color greenyellow sim:/mod10counter_tb/uut/currentCount
```

8.5 FPGA: Do file

You have been required to run simulation for almost every lab. This is not busy-work, all practicing digital designers are expected to demonstrate proper functionality before modules are incorporated into larger designs. You may have experienced times when you had to run a simulation multiple times to get your design correct. You probably got tired setting up all the signals, colors and radix every time you found an error. Good news! You can build a script that does this setup automatically, a do file. A do file is a script to setup the waveforms for a simulation including their order, colors and radixes.

Listing 1 shows a partial do file for the `mod10counter` testbench. The "#" symbol is used to denote comments – any text placed after them is ignored by the do file interpreter.

The first three lines start the simulation and delete any waveforms that may be left over from a previous simulation. I included this in the do file because I sometimes rerun a simulation to observe the modules behavior at some earlier time.

The waveforms in the simulation are added using the "add wave" command. If you drag-and-drop a signal name into the waveform area, you will see the add wave command appear in the console are at the bottom of the Questa window. For example when I manually added the `clk`, I see:

```
add wave -position end sim:/mod10counter_tb/uut/clk
```

I then add the color of the wave and the radix in between "end" and "sim". See Listing 1 for a couple of examples. You should use notepad to edit the do file. Do NOT use a word processor like MS Word because they tend to change the extensions of files when you save.

After editing the do file, it should be stored in the following folder:

```
<projectDirectory>\mod10Counter\simulation\questa
```

Where <projectDirectory> is the path of your project. If this folder does not exit, then you need to have to run "Start Analysis and Elaboration" at least once. You know you are in the project directory when you see the project QPF file.

When you have the testbench setup in Questa, you will execute the commands in the do file by typing the following into the console window. The console is at the bottom of the Questa window.

```
do mod10counter_tbWaveSetup.do
```

The Questa console allows tab completion, a feature that helps you fill-in the characters of long file name. To use tab completion, type the first few characters of a command/filename and then press Tab. If there are no other file names that match what you have typed in, the

remainder of the file name will be auto-complete for you. If there is more than one filename choice, the command/filename will be completed up to the ambiguity and the console will provide a list of candidate filenames.

You can issue a variety of commands in the console window. A very popular command is "run <time>" to simulate some amount of time.

8.6 Testbench

Edit the do file provided on Canvas to produce the following output.

signal	radix	signal color
`clk`	default	green
`reset`	default	green
`enb`	default	gold
`synch`	default	gold
`roll`	default	yellow
`currentCount`	unsigned	greenyellow

You need to look at the values produced by the `mod10counter` and compare them against the values in Table 8.1. Look for discrepancies starting at time 0 and only advancing the simulation when everything is correct. You will have to transcend the design hierarchy to find the source of your errors. Most errors are due to incorrect wiring or modules – wrong names or wrong signal order.

8.7 Turn in

You may work in teams of at most two. Make a record of your response to the items below and turn them in a single copy as your team's solution on Canvas using the instructions posted there. Include the names of both team members at the top of your solutions. Use complete English sentences to introduce what each of the following listed items (below) is and how it was derived. In addition to this submission, you will be expected to demonstrate your circuit at the beginning of your lab section next week.

Mod10 Counter Verilog

- Complete the truth table in Table 8.3.
- Complete the hand-drawn timing diagram in Figure 8.3.
- Verilog code for the body of the mod10counter module (courier 8-point font single spaced), leave out header comments.

Testbench

- Complete simulation timing diagram of the `mod10counter`.
- The contents of your Do File.